

Data Governance & Quality

Strategies in ETL / ELT Data Engineering

A comprehensive technical guide covering every governance strategy
applied across Bronze · Silver · Gold · Consumption layers

with PySpark & SQL code examples

May 2026 | Data Engineering Series

Table of Contents

1. Introduction to Data Governance in Data Engineering	3
2. Bronze Layer — Ingestion Governance	4
2.1. Metadata Stamping	4
2.2. Append-Only Raw Preservation	5
2.3. Watermarking & Duplicate Ingestion Guard	5
3. Silver Layer — Transformation Governance	7
3.1. Schema Enforcement	7
3.2. Null & Range Validation (Data Quality Checks)	8
3.3. Deduplication	9
3.4. PII Masking & Anonymization	10
3.5. Data Standardization & Normalization	11
3.6. Data Type Casting	12
3.7. Audit Column Injection	13
3.8. Bad Record Quarantine (Dead Letter Pattern)	14
3.9. SCD — Slowly Changing Dimensions	15
4. Gold Layer — Business Logic Governance	17
4.1. Business Rule Enforcement	17
4.2. Row-Level Security (RLS)	18
4.3. Metric Consistency & Data Contracts	19
5. Consumption Layer — Access Governance	20
5.1. Column-Level Security & Dynamic Masking	20
5.2. RBAC — Role-Based Access Control	21
5.3. Data Retention & Lifecycle Policies	22
6. Pipeline / Orchestration Governance	23
6.1. Data Lineage Tracking	23
6.2. SLA Monitoring & Alerting	24
6.3. Environment Segregation (Dev/Stage/Prod)	25
7. Quick-Reference Summary Tables	26

Introduction to Data Governance in Data Engineering

Data Governance in data engineering is the practice of enforcing policies, rules, and standards across every stage of a data pipeline — from raw ingestion all the way to consumption by analysts and applications. It is NOT a one-time activity but an ongoing, automated discipline embedded inside ETL/ELT code.

In the Medallion Architecture (Bronze → Silver → Gold), governance is applied differently at each layer. Each layer has specific responsibilities, and governance strategies are designed to meet those responsibilities.

Layer	Governance Focus	Primary Goal
Bronze	Capture safely	Preserve raw data, stamp metadata, encrypt
Silver	Transform safely	Clean, validate, mask PII, deduplicate
Gold	Aggregate safely	Business rules, RLS, metric consistency
Consumption	Access safely	RBAC, column masking, compliance
Pipeline	Operate safely	Lineage, SLAs, CI/CD controls

Key Principle: Silver is the most intensive governance layer, but governance is an END-TO-END responsibility — applied from first byte ingested to last query consumed.

Bronze Layer — Ingestion Governance

The Bronze layer receives raw data exactly as-is from source systems. Governance here focuses on safe capture — not transformation. The guiding rule is: never lose or alter the original data.

2.1 Metadata Stamping

Every record ingested into Bronze must carry audit metadata so we know WHERE the data came from, WHEN it arrived, and from WHICH file or system. This is essential for debugging, lineage tracking, and compliance audits.

Scenario: A financial company ingests transactions from 5 different banks. Six months later an audit asks 'which bank sent this duplicate transaction and when?'. Without metadata stamps, you cannot answer. With them, it is a single query.

PySpark Code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, lit

df = spark.read.json('s3://raw/transactions/*.json')

# Stamp governance metadata on every record

df = df \

.withColumn('_ingested_at', current_timestamp()) \

.withColumn('_source_file', input_file_name()) \

.withColumn('_source_system', lit('BANK_FEEDS')) \

.withColumn('_pipeline_ver', lit('v1.4.2'))

df.write.mode('append').parquet('bronze/transactions')
```

SQL (dbt Bronze Model):

```
-- models/bronze/transactions.sql

SELECT

*,

CURRENT_TIMESTAMP() AS _ingested_at,
```

```
'{{ var("source_system") }}' AS _source_system,  
  
'{{ var("pipeline_version") }}' AS _pipeline_ver  
  
FROM {{ source('raw', 'transactions') }}
```

2.2 Append-Only Raw Preservation

Bronze should be write-once, read-many. Raw data must never be updated or deleted after ingestion. This guarantees you can always replay the full pipeline from scratch if a bug is found in a downstream transformation.

Scenario: A data engineer accidentally applies wrong currency conversion logic in Silver. Because Bronze is append-only and untouched, they simply re-run the Silver job from Bronze and the data is corrected — no data is permanently lost.

PySpark Code:

```
# ALWAYS use append mode for Bronze — NEVER overwrite  
  
df.write \  
  
.mode('append') \ # <-- governance rule: never 'overwrite'  
  
.partitionBy('year', 'month', 'day') \  
  
.parquet('bronze/transactions')  
  
# Protect the path via IAM / storage policy:  
  
# Only the ingestion service account can WRITE to bronze/  
  
# All other roles get READ-ONLY access
```

2.3 Watermarking & Duplicate Ingestion Guard

Track the last successfully ingested timestamp or offset. On each run, only fetch NEW records. This prevents re-ingesting the same data twice (which would create duplicates in Bronze) and makes pipelines idempotent.

Scenario: An Airflow pipeline crashes mid-run and is retried automatically. Without a watermark, it re-ingests everything from the beginning. With a watermark, it continues exactly from where it stopped.

PySpark Code:

```
# Read the last watermark from a control table  
  
wm = spark.read.parquet('meta/watermarks') \  

```

```

.filter(col('table_name') == 'transactions') \

.select('last_loaded_at').first()[0]

# Fetch only new records

df = spark.read.jdbc(

url=jdbc_url,

table='transactions',

predicates=[f"updated_at > '{wm}'"]

)

# After successful write, update the watermark

new_wm = df.agg({'updated_at': 'max'}).first()[0]

spark.createDataFrame([('transactions', new_wm)], ['table_name', 'last_loaded_at']) \

.write.mode('overwrite').parquet('meta/watermarks')

```

SQL (Watermark Check):

```

-- Pull only records newer than last watermark

SELECT *

FROM source_transactions

WHERE updated_at > (

SELECT MAX(last_loaded_at)

FROM pipeline_control.watermarks

WHERE table_name = 'transactions'

)

```

Silver Layer — Transformation Governance

Silver is the most governance-intensive layer. Raw Bronze data is cleaned, validated, masked, standardized, and enriched here. Only data that passes all governance checks should leave Silver and enter Gold.

3.1 Schema Enforcement

Enforce a strict, predefined schema when reading from Bronze. Reject or flag any records that do not match. This prevents schema drift — where upstream source systems silently change column names, types, or add/remove fields — from corrupting downstream tables.

Scenario: An e-commerce source system renames 'cust_id' to 'customer_id' after a replatforming. Without schema enforcement, Silver joins silently start producing nulls. With enforcement, the pipeline immediately fails and alerts the team.

PySpark Code:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, TimestampType

# Define the expected Silver schema as a contract

silver_schema = StructType([

    StructField('customer_id', StringType(), nullable=False),

    StructField('email', StringType(), nullable=True),

    StructField('age', IntegerType(), nullable=True),

    StructField('created_at', TimestampType(), nullable=False),

])

# This will raise an AnalysisException if Bronze schema drifts

df = spark.read.schema(silver_schema).parquet('bronze/customers')

# Or use PERMISSIVE mode + capture bad records

df = spark.read \

    .option('mode', 'PERMISSIVE') \

    .option('columnNameOfCorruptRecord', '_corrupt_record') \
```

```
.schema(silver_schema) \  
  
.parquet('bronze/customers')
```

SQL / dbt Schema Test:

```
-- schema.yml – schema as a governance contract  
  
models:  
  
  - name: silver_customers  
  
    columns:  
  
      - name: customer_id  
  
        data_type: string  
  
        tests:  
  
          - not_null  
  
          - unique  
  
          - name: created_at  
  
            data_type: timestamp  
  
            tests:  
  
              - not_null
```

3.2 Null & Range Validation (Data Quality Checks)

Check data against business rules at the record level: mandatory fields must not be null, numeric values must fall within valid ranges, and categorical fields must contain only allowed values. Records that fail are routed to quarantine (see 3.8).

Scenario: A healthcare pipeline receives patient age as -5 or 999 due to a source system bug. Without range checks, these corrupt values feed into risk models and produce incorrect clinical decisions.

PySpark Code:

```
from pyspark.sql.functions import col, when  
  
# Null check – required fields  
  
df_valid = df.filter(col('customer_id').isNotNull())  
  
# Range validation
```



```

df_valid = df_valid.filter(

(col('age') >= 0) & (col('age') <= 120)

)

# Accepted values check

allowed_status = ['ACTIVE', 'INACTIVE', 'PENDING']

df_valid = df_valid.filter(col('status').isin(allowed_status))

# Regex validation (email format)

df_valid =
df_valid.filter(col('email').rlike(r'^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$'))

```

SQL (dbt tests + inline validation):

```

-- Inline in transformation model

SELECT *

FROM bronze_customers

WHERE customer_id IS NOT NULL

AND age BETWEEN 0 AND 120

AND status IN ('ACTIVE', 'INACTIVE', 'PENDING')

AND email LIKE '%@%.%'

-- dbt schema test for accepted values

# schema.yml

- name: status

tests:

- accepted_values:

values: ['ACTIVE', 'INACTIVE', 'PENDING']

```

3.3 Deduplication

Source systems often send duplicate records due to network retries, CDC edge cases, or manual re-sends. Deduplication ensures each logical entity (customer, order, event) appears exactly once in Silver, using a defined business key.

Scenario: A Kafka consumer processes an order event twice because of a network timeout causing an at-least-once delivery. Without deduplication, the order is counted twice in revenue reports — a direct financial error.

PySpark Code:

```
from pyspark.sql.functions import col, row_number

from pyspark.sql.window import Window

# Simple dedup – keep first occurrence by business key
df_deduped = df.dropDuplicates(['order_id'])

# Advanced dedup – keep the LATEST version of each record
window = Window.partitionBy('order_id').orderBy(col('updated_at').desc())

df_deduped = df \

.withColumn('row_num', row_number().over(window)) \

.filter(col('row_num') == 1) \

.drop('row_num')
```

SQL Code:

```
-- Keep latest record per order_id using ROW_NUMBER()

WITH ranked AS (

SELECT *,

ROW_NUMBER() OVER (

PARTITION BY order_id

ORDER BY updated_at DESC

) AS row_num

FROM bronze_orders

)

SELECT * EXCLUDE (row_num)

FROM ranked

WHERE row_num = 1
```

3.4 PII Masking & Anonymization

Personally Identifiable Information (PII) such as email addresses, phone numbers, national IDs, and credit card numbers must be masked, hashed, or removed before data reaches Silver. This is mandatory for GDPR, HIPAA, PCI-DSS compliance.

Scenario: A retail company stores customer emails in Bronze. Analysts need to analyze customer behavior but should NOT see raw emails. The Silver layer hashes emails with SHA-256 — analysts can still join on the hash, but cannot reverse it to a real email.

PySpark Code:

```
from pyspark.sql.functions import sha2, col, lit, concat_ws, regexp_replace

# Hash PII — SHA-256 is irreversible

df = df.withColumn('email_hash', sha2(col('email'), 256))

# Mask phone: show only last 4 digits

df = df.withColumn('phone_masked',

concat_ws('-', lit('***-***-'), col('phone').substr(-4, 4)))

# Mask credit card number

df = df.withColumn('card_masked',

regexp_replace(col('card_number'), r'\d(?:\d{4})', '*'))

# Drop the original PII columns completely

df = df.drop('email', 'phone', 'card_number')
```

SQL Code:

```
SELECT

customer_id,

SHA2(email, 256) AS email_hash,

CONCAT('***-***-', RIGHT(phone, 4)) AS phone_masked,

REGEXP_REPLACE(card_number, '[0-9](?=[0-9]{4})', '*') AS card_masked,

age,

created_at

FROM bronze_customers
```

```
-- email, phone, card_number are NOT selected (dropped from Silver)
```

3.5 Data Standardization & Normalization

Standardize text casing, trim whitespace, unify date formats, and replace implicit nulls (empty strings, 'N/A', 'NULL') with proper SQL NULLs. This ensures consistent joins and aggregations across data from different source systems.

Scenario: A customer master receives 'USA', 'usa', 'United States', ' US ' from four different source systems. Without standardization, country-level reports show 4 separate rows instead of one — breaking every dashboard built on that data.

PySpark Code:

```
from pyspark.sql.functions import upper, lower, trim, to_timestamp, when, col

df = df \

.withColumn('country', upper(trim(col('country')))) \

.withColumn('status', lower(trim(col('status')))) \

.withColumn('first_name', initcap(trim(col('first_name')))) \

.withColumn('created_at', to_timestamp(col('created_at'), 'yyyy-MM-dd HH:mm:ss')) \

.withColumn('revenue', col('revenue').cast('decimal(18,2)'))

# Replace blank strings and 'N/A' with proper NULL

for field in ['email', 'phone', 'address']:

df = df.withColumn(field,

when(col(field).isin(['', 'N/A', 'NULL', 'none']), None)

.otherwise(col(field)))
```

SQL Code:

```
SELECT

UPPER(TRIM(country)) AS country,

LOWER(TRIM(status)) AS status,

INITCAP(TRIM(first_name)) AS first_name,

TRY_CAST(created_at AS TIMESTAMP) AS created_at,

NULLIF(NULLIF(TRIM(email), ''), 'N/A') AS email
```

```
FROM bronze_customers
```

3.6 Data Type Casting

Bronze often stores everything as strings (especially from JSON/CSV sources). Silver must cast each column to its correct data type. Use safe casting to avoid pipeline failures on malformed values — instead, bad casts produce NULL and get quarantined.

Scenario: Revenue values come in as strings like '1,234.56' or '1234.56 USD'. Without type casting, a `SUM(revenue)` query returns an error or silently returns 0. Proper casting ensures financial aggregations are numerically accurate.

PySpark Code:

```
from pyspark.sql.functions import col, regexp_replace

from pyspark.sql.types import DecimalType, IntegerType, DateType, BooleanType

# Strip non-numeric chars before casting revenue

df = df.withColumn('revenue_clean',
    regexp_replace(col('revenue'), '[^0-9.]', ''))

# Safe casting – invalid values become NULL instead of crashing

df = df \

    .withColumn('revenue', col('revenue_clean').cast(DecimalType(18, 2))) \

    .withColumn('quantity', col('quantity').cast(IntegerType())) \

    .withColumn('order_date', col('order_date').cast(DateType())) \

    .withColumn('is_active', col('is_active').cast(BooleanType()))

df = df.drop('revenue_clean')
```

SQL Code:

```
SELECT

TRY_CAST(REGEXP_REPLACE(revenue, '[^0-9.]', '') AS DECIMAL(18,2)) AS revenue,

TRY_CAST(quantity AS INT) AS quantity,

TRY_CAST(order_date AS DATE) AS order_date,

TRY_CAST(is_active AS BOOLEAN) AS is_active
```

```
FROM bronze_orders
```

```
-- TRY_CAST returns NULL on failure rather than raising an error
```

3.7 Audit Column Injection

Every record in Silver and beyond should carry standard audit columns. These enable debugging, compliance reporting, and pipeline monitoring without needing to join back to logs. They are injected by the pipeline — not sourced from the upstream system.

Scenario: A compliance officer asks 'show me every record that was modified in the last 30 days, which pipeline touched it, and what version ran'. Audit columns make this a trivial query. Without them, it requires cross-referencing multiple log systems.

Standard Audit Columns:

Column	Type	Purpose
_ingested_at	TIMESTAMP	When the record entered the pipeline
_processed_at	TIMESTAMP	When the Silver transformation ran
_source_system	STRING	Which source system provided this record
_pipeline_version	STRING	Version of the ETL code that processed it
_bronze_path	STRING	Source file/path in Bronze layer

PySpark Code:

```
from pyspark.sql.functions import current_timestamp, lit

df = df \

.withColumn('_processed_at', current_timestamp()) \

.withColumn('_source_system', lit('CRM_SALESFORCE')) \

.withColumn('_pipeline_version', lit('v2.3.1')) \

.withColumn('_bronze_path', lit('bronze/customers/2026/05/'))
```

SQL (dbt macro):

```
-- macros/audit_columns.sql

{% macro audit_columns(source_system, pipeline_version) %}

CURRENT_TIMESTAMP() AS _processed_at,

'{{ source_system }}' AS _source_system,
```

```

'{{ pipeline_version }}' AS _pipeline_version

{% endmacro %}

-- Usage in any Silver model

SELECT *, {{ audit_columns('CRM_SALESFORCE', 'v2.3.1') }}

FROM {{ ref('bronze_customers') }}

```

3.8 Bad Record Quarantine (Dead Letter Pattern)

Instead of failing the entire pipeline on bad records, split data into GOOD and BAD streams. Good records proceed to Silver. Bad records go to a quarantine zone with a reason tag. This keeps pipelines running while providing full visibility into data quality issues for investigation and replay.

Scenario: 2% of daily customer records have null emails. Failing the whole pipeline blocks 98% of valid data. Quarantine pattern lets the 98% flow through, while the 2% is stored with a reason code for the source team to fix and resubmit.

PySpark Code:

```

from pyspark.sql.functions import col, lit, current_timestamp

# Define validation conditions

valid_mask = (

col('customer_id').isNotNull() &

col('email').contains('@') &

col('age').between(0, 120)

)

# Split into good and bad

good_df = df.filter(valid_mask)

bad_df = df.filter(~valid_mask)

# Tag quarantine records with reason and timestamp

bad_df = bad_df \

.withColumn('_quarantine_reason', lit('FAILED_VALIDATION')) \

.withColumn('_quarantine_at', current_timestamp())

```

```
# Write bad records to quarantine zone

bad_df.write.mode('append').parquet('quarantine/customers')

# Only good records proceed to Silver

good_df.write.mode('overwrite').parquet('silver/customers')

print(f'Valid: {good_df.count()} | Quarantined: {bad_df.count()}')
```

SQL Code:

```
-- Route valid records to Silver

INSERT INTO silver.customers

SELECT * FROM bronze_customers

WHERE customer_id IS NOT NULL

AND email LIKE '%@%'

AND age BETWEEN 0 AND 120;

-- Route bad records to quarantine

INSERT INTO quarantine.customers

SELECT *, 'FAILED_VALIDATION' AS _reason, CURRENT_TIMESTAMP() AS _quarantine_at

FROM bronze_customers

WHERE customer_id IS NULL

OR email NOT LIKE '%@%'

OR age NOT BETWEEN 0 AND 120;
```

3.9 SCD — Slowly Changing Dimensions

Dimension data (customers, products, employees) changes slowly over time. SCD Type 1 overwrites old values. SCD Type 2 keeps full history by versioning rows with `effective_from` / `effective_to` dates. This is critical for accurate historical reporting.

Scenario: A customer changes their billing address. SCD Type 1 overwrites it — orders from last year now show the new address, breaking historical analysis. SCD Type 2 preserves both old and new addresses with date ranges, keeping history intact.

PySpark / Delta Lake MERGE (SCD Type 2):


```

from delta.tables import DeltaTable

from pyspark.sql.functions import current_timestamp, lit

silver = DeltaTable.forPath(spark, 'silver/customers')

silver.alias('existing').merge(
    incoming_df.alias('new'),
    'existing.customer_id = new.customer_id AND existing.is_current = true'
).whenMatchedUpdate(
    condition='existing.address != new.address', # detect change
    set={
        'is_current': lit(False),
        'effective_to': current_timestamp()
    }
).whenNotMatchedInsertAll() \

.execute()

# Insert new version of changed records

changed = incoming_df.join(silver.toDF(), 'customer_id', 'inner') \

.filter('...address changed...')

changed.withColumn('is_current', lit(True)) \

.withColumn('effective_from', current_timestamp()) \

.withColumn('effective_to', lit(None)) \

.write.mode('append').format('delta').save('silver/customers')

```

SQL (Snowflake MERGE for SCD Type 2):

```

MERGE INTO silver.customers AS target

USING staging.customers AS source

ON target.customer_id = source.customer_id AND target.is_current = TRUE

WHEN MATCHED AND target.address != source.address THEN

UPDATE SET

is_current = FALSE,

```

```
effective_to = CURRENT_TIMESTAMP()

WHEN NOT MATCHED THEN

INSERT (customer_id, name, address, is_current, effective_from, effective_to)

VALUES (source.customer_id, source.name, source.address,

TRUE, CURRENT_TIMESTAMP(), NULL);
```

Gold Layer — Business Logic Governance

Gold layer transforms clean Silver data into business-ready aggregations and metrics. Governance here ensures business rules are consistently enforced, KPIs have a single definition, and access is controlled at the row and column level.

4.1 Business Rule Enforcement

Critical business logic — revenue recognition, order status definitions, date ranges for reporting periods — must be encoded ONCE in the Gold model and never duplicated. This is the single source of truth principle.

Scenario: Finance, Sales, and Marketing each compute monthly revenue differently — different date filters, different status inclusions. Gold enforces one definition: 'completed orders within the calendar month'. All teams use the same Gold table.

PySpark Code:

```
from pyspark.sql.functions import col, sum, countDistinct, when

# Single definition of 'valid revenue' enforced in Gold

gold_revenue = silver_orders \

.filter(col('status') == 'COMPLETED') \

.filter(col('order_date').between('2026-01-01', '2026-01-31')) \

.groupBy('region', 'product_category') \

.agg(

    sum('revenue').alias('total_revenue'),

    countDistinct('customer_id').alias('unique_customers'),

    sum(when(col('is_new_customer'), 1).otherwise(0)).alias('new_customers')

)

gold_revenue.write.mode('overwrite').parquet('gold/revenue_summary')
```

SQL (dbt Gold model):

```
-- models/gold/revenue_summary.sql
```

```
-- {{ config(materialized='table', tags=['gold','finance']) }}

SELECT

region,

product_category,

DATE_TRUNC('month', order_date) AS report_month,

SUM(revenue) AS total_revenue,

COUNT(DISTINCT customer_id) AS unique_customers

FROM {{ ref('silver_orders') }}

WHERE status = 'COMPLETED' -- business rule enforced here once

GROUP BY 1, 2, 3
```

4.2 Row-Level Security (RLS)

Row-Level Security ensures that when multiple teams query the same Gold table, each user or role automatically sees only the rows they are authorized to see — without requiring separate tables or manual query filters.

Scenario: A global retail company has regional managers in EMEA, APAC, and Americas. All three query gold.sales_summary, but each should only see their own region's data. RLS enforces this transparently at the database layer.

SQL (Snowflake Row Access Policy):

```
-- Create a row access policy

CREATE OR REPLACE ROW ACCESS POLICY region_access_policy

AS (region VARCHAR) RETURNS BOOLEAN ->

CURRENT_ROLE() IN ('ADMIN', 'DATA_ENGINEER')

OR CURRENT_USER() = region || '_MANAGER';

-- Attach to Gold table

ALTER TABLE gold.sales_summary

ADD ROW ACCESS POLICY region_access_policy ON (region);

-- EMEA manager only sees EMEA rows; query is same for all users

SELECT * FROM gold.sales_summary; -- RLS filters transparently
```

PySpark Code (programmatic RLS):

```
# Apply user-level filter before returning data

def apply_qls(df, user_role, user_region):

    if user_role == 'ADMIN':

        return df # full access

    return df.filter(col('region') == user_region)

df_gold = spark.read.parquet('gold/sales_summary')

df_user = apply_qls(df_gold, current_user_role, current_user_region)
```

4.3 Metric Consistency & Data Contracts

A Data Contract is a formal agreement between the team producing a Gold dataset and the teams consuming it. It specifies: schema, column definitions, SLAs (freshness, row count ranges), and data quality thresholds. Violations trigger alerts.

Data Contract YAML Example:

```
# data_contract_revenue_summary.yaml

dataContractSpecification: 0.9.3

id: revenue-summary-v1

info:

    title: Revenue Summary

    owner: data-engineering-team

    sla:

        freshness: '2 hours'

        row_count_min: 1000

models:

    revenue_summary:

        fields:

            total_revenue:

                type: decimal
```

```
required: true

minimum: 0

report_month:

type: date

required: true

quality:

- type: sql

query: SELECT COUNT(*) FROM gold.revenue_summary WHERE total_revenue < 0

must_be: 0

description: 'No negative revenue allowed'
```

Consumption Layer — Access Governance

The consumption layer is where analysts, data scientists, and applications query Gold tables. Governance here focuses on controlling access to sensitive data through dynamic masking, RBAC, and enforcing retention policies.

5.1 Column-Level Security & Dynamic Masking

Different roles see different representations of the same column. An ADMIN sees the real email; an ANALYST sees '***masked***'. The same table serves both — no duplicate tables needed. This is enforced at the warehouse layer.

SQL (Snowflake Masking Policy):

```
-- Define masking policy for email column

CREATE OR REPLACE MASKING POLICY email_mask

AS (val STRING) RETURNS STRING ->

CASE

WHEN CURRENT_ROLE() IN ('DATA_ENGINEER', 'ADMIN') THEN val

WHEN CURRENT_ROLE() = 'ANALYST' THEN CONCAT(LEFT(val,2), '***@***.com')

ELSE '***MASKED***'

END;

-- Attach masking policy to column

ALTER TABLE gold.customers

MODIFY COLUMN email

SET MASKING POLICY email_mask;

-- ANALYST query sees: jo***@***.com

-- ADMIN query sees: john.doe@company.com
```

5.2 RBAC — Role-Based Access Control

Define roles with minimum necessary privileges. Assign users to roles, not to individual tables. This ensures the principle of least privilege and simplifies access management as teams grow.

SQL (Snowflake RBAC setup):

```
-- Create role hierarchy

CREATE ROLE data_engineer;

CREATE ROLE analyst;

CREATE ROLE viewer;

-- Grant privileges

GRANT ALL PRIVILEGES ON DATABASE prod_db TO ROLE data_engineer;

GRANT SELECT ON ALL TABLES IN SCHEMA gold TO ROLE analyst;

GRANT SELECT ON TABLE gold.revenue_summary TO ROLE viewer;

-- Assign users to roles

GRANT ROLE analyst TO USER alice;

GRANT ROLE viewer TO USER bob;

-- Analysts CANNOT read Silver PII tables

REVOKE SELECT ON SCHEMA silver FROM ROLE analyst;
```

5.3 Data Retention & Lifecycle Policies

Data must not be kept indefinitely. GDPR mandates deletion on request. HIPAA requires 7-year retention. Define and automate retention policies per dataset. Implement 'right to be forgotten' by deleting or anonymizing a user's records across all layers.

SQL (Delete expired records):

```
-- GDPR: right to be forgotten – delete customer across all layers

DELETE FROM gold.customers WHERE customer_id = '{{cust_id}}';

DELETE FROM silver.customers WHERE customer_id = '{{cust_id}}';

DELETE FROM silver.orders WHERE customer_id = '{{cust_id}}';

-- Retention: purge records older than 7 years

DELETE FROM silver.transactions
```



```
WHERE created_at < DATEADD('year', -7, CURRENT_DATE());
```

PySpark Code (Delta Lake delete):

```
from delta.tables import DeltaTable

# GDPR right-to-be-forgotten

customer_id_to_delete = 'CUST_12345'

for table_path in ['silver/customers', 'silver/orders', 'gold/customers']:

    dt = DeltaTable.forPath(spark, table_path)

    dt.delete(f"customer_id = '{customer_id_to_delete}'")

# Purge old data (retention policy)

dt = DeltaTable.forPath(spark, 'silver/transactions')

dt.delete("created_at < date_sub(current_date(), 2555)") # 7 years
```

Pipeline / Orchestration Governance

Pipeline governance cuts across all layers and is enforced at the orchestration level (Airflow, dbt Cloud, etc.). It includes lineage tracking, SLA monitoring, and environment segregation to prevent production incidents.

6.1 Data Lineage Tracking

Data lineage records which datasets feed into which, and what transformations were applied. When a source field changes, lineage lets you instantly identify every downstream table, report, and dashboard that is affected.

OpenLineage Integration (PySpark):

```
# OpenLineage emits lineage events automatically during Spark jobs

# Configure in spark-defaults.conf or SparkSession:

spark = SparkSession.builder \

    .config('spark.extraListeners',

        'io.openlineage.spark.agent.OpenLineageSparkListener') \

    .config('spark.openlineage.transport.type', 'http') \

    .config('spark.openlineage.transport.url', 'http://marquez:5000') \

    .config('spark.openlineage.namespace', 'data-engineering') \

    .getOrCreate()

# Every read/write now emits lineage to Marquez automatically

df = spark.read.parquet('bronze/orders')

df.write.parquet('silver/orders')

# -> lineage: bronze/orders -> transform_orders_job -> silver/orders
```

6.2 SLA Monitoring & Alerting

Define SLAs (Service Level Agreements) for each pipeline — e.g., 'Silver customers must be ready by 7:00 AM every day'. Airflow monitors these and triggers alerts to the team if a pipeline misses its SLA, so downstream consumers are proactively notified.

Airflow SLA Configuration:

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta

def notify_sla_miss(dag, task_list, blocking_task_list, slas, blocking_tis):

    # Send alert to Slack/PagerDuty/email

    send_slack_alert(f'SLA MISSED: {dag.dag_id} – downstream tables may be delayed')

with DAG(

    dag_id='silver_customer_pipeline',

    schedule_interval='0 5 * * *', # run at 5 AM

    sla_miss_callback=notify_sla_miss,

    default_args={

        'sla': timedelta(hours=2), # must finish by 7 AM

        'retries': 3,

        'retry_delay': timedelta(minutes=5),

    },

    catchup=False

) as dag:

    transform_task = PythonOperator(

        task_id='transform_silver_customers',

        python_callable=run_silver_transform

    )
```

6.3 Environment Segregation (Dev / Staging / Prod)

Strictly separate pipeline environments. Development and staging should never contain real PII. Use masked/synthetic data in non-production. Changes must pass CI/CD checks (schema tests, data quality tests) before deploying to production.

dbt Environment Config:

```
# dbt_project.yml – separate targets per environment
```

```
models:

my_project:

+schema: '{{ target.schema }}' # dev_silver, staging_silver, silver

# profiles.yml

my_project:

target: dev

outputs:

dev:

schema: dev_silver

# uses anonymized dev data

staging:

schema: staging_silver

# masked copy of prod data

prod:

schema: silver

# real prod data, restricted access

# CI/CD pipeline (GitHub Actions) – runs tests before prod deploy

# dbt test --target staging --> must pass before merge to main
```

Quick-Reference Summary Tables

Complete Strategy Reference — All Layers

Strategy	Layer	Why It Matters	Key Tool / Function
Metadata Stamping	Bronze	Traceability & lineage	<code>current_timestamp()</code> , <code>lit()</code>
Append-Only Preservation	Bronze	Data replayability	<code>write.mode('append')</code>
Watermarking	Bronze	Dedup ingestion, idempotency	Control table + predicates
Schema Enforcement	Silver	Prevent schema drift	<code>StructType</code> , <code>dbt schema.yml</code>
Null & Range Validation	Silver	Data accuracy & trust	<code>filter()</code> , <code>dbt tests</code>
Deduplication	Silver	Avoid double-counting	<code>dropDuplicates()</code> , <code>ROW_NUMBER()</code>
PII Masking	Silver	GDPR/HIPAA compliance	<code>sha2()</code> , <code>regexp_replace()</code>
Standardization	Silver	Consistent joins & aggregations	<code>upper()</code> , <code>trim()</code> , <code>NULLIF()</code>
Type Casting	Silver	Correct aggregations	<code>cast()</code> , <code>TRY_CAST()</code>
Audit Columns	Silver/Gold	Debugging & compliance	<code>current_timestamp()</code> , <code>lit()</code>
Quarantine Pattern	Silver	Keep pipelines running	Split filter + dead letter
SCD Type 2	Silver	Preserve dimension history	Delta <code>MERGE</code> , <code>MERGE INTO</code>
Business Rule Enforcement	Gold	Single metric definition	<code>dbt models</code> , <code>filter()</code>
Row-Level Security	Gold	Data access control	<code>ROW ACCESS POLICY</code>
Data Contracts	Gold	Producer-consumer trust	<code>data-contract-cli</code> , <code>YAML</code>
Column Masking	Consumption	PII protection at query time	<code>MASKING POLICY</code>
RBAC	Consumption	Least privilege access	<code>GRANT/REVOKE</code> , <code>IAM</code>
Retention Policies	Consumption	Legal compliance (GDPR/HIPAA)	<code>DELETE</code> , <code>dt.delete()</code>
Lineage Tracking	Pipeline	Impact analysis	<code>OpenLineage</code> , <code>dbt lineage</code>
SLA Monitoring	Pipeline	Operational reliability	Airflow <code>SLA callbacks</code>
Env Segregation	Pipeline	Prevent prod incidents	<code>dbt targets</code> , <code>CI/CD</code>

Compliance Framework Coverage

Regulation	Key Requirement	Strategies Applied
GDPR	Right to erasure, data minimization, consent	PII Masking, Retention Policies, RBAC, Column Masking
HIPAA	PHI protection, audit trails, 7-yr retention	PII Masking, Audit Columns, Retention Policies, RBAC
PCI-DSS	Cardholder data protection, access control	PII Masking (card), Column Masking, RBAC, Env Segregation
SOX	Financial data accuracy, audit trails	Audit Columns, Lineage, Business Rule Enforcement

Final Takeaway: Data Governance is not a bolt-on — it is embedded in every line of transformation code. The most resilient pipelines treat governance as a first-class engineering requirement, automated in CI/CD, and monitored continuously in production.